FIRST VALKEY MEETUP · DELHI · JUNE 2025

Rate Limiting *at Scale*

From Redis Patterns to Valkey — and Into the AI Era

Prabal · Senior Backend Engineer

7+ years building distributed systems at scale

Stealth Startup · Remote

AGENDA

20 minutes, one clear argument.

01 Why naive rate limiting fails at scale

02 The three algorithms — and when to use each

03 Valkey's atomic Lua scripts: the unfair advantage

04 Valkey 9.0: hash field TTL kills key sprawl

05 The AI angle — rate limiting by token count, not requests

06 Failure modes, persistence tradeoffs + live demo

THE PROBLEM

Why naive rate limiting breaks.

- **Split counters**

  Multiple servers each hold partial counts. User A can make 3× the allowed requests across 3 pods.

- **Race conditions**

  Two threads read count=99, both increment to 100 — only one should have been allowed.

- **Window boundary burst**

  User sits at limit=0 at 11:59:59. Window resets. They fire 2× limit in 2 seconds.

- **No atomicity**

  Check-then-set is never safe. The gap between read and write is where abuse lives.

The naive approach

```
# Per-pod in-memory counter
count = cache.get(user_id)
if count > LIMIT:
    raise RateLimitError()

# What actually happens:
Thread 1: reads count=99
Thread 2: reads count=99
Thread 1: writes count=100  # allowed
Thread 2: writes count=100  # also allowed

# Double-spend. Every time.
```

THE ALGORITHMS

Choosing the right algorithm.

Fixed Window

SIMPLE TO IMPLEMENT

Boundary burst: users can double their limit in 2 seconds at the reset edge.

```
INCR key
EXPIRE key 60
```

Internal tooling, low-stakes endpoints

Avoid for public APIs

Sliding Window

INDUSTRY STANDARD

2× memory (stores timestamps). Counts window accurately with ZADD + ZREMRANGEBYSCORE.

```
ZADD key now member
ZREMRANGEBYSCORE key
ZCARD key
```

Public REST APIs, user-facing rate limits

Default choice

Token Bucket

BURST-FRIENDLY, COMPLEX

Allows controlled bursting up to capacity. Refills at constant rate. Requires Lua for atomicity.

```
HGET tokens
HGET last_refill
# Lua atomic refill
```

LLM APIs, SDKs, metered billing

Best for AI workloads

VALKEY ADVANTAGE

Lua scripts: atomicity without locks.

1 **round-trip**

Check + increment + expire.
Atomic. No race. No lock.

22% lower P99 latency vs Redis OSS
1.2M ops/sec on Valkey 8.1

Sliding window — Lua (fully atomic)

```lua
local key    = KEYS[1]
local now    = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit  = tonumber(ARGV[3])

-- Drop expired entries
redis.call('ZREMRANGEBYSCORE',
  key, 0, now - window)

local count = redis.call('ZCARD', key)

if count < limit then
  redis.call('ZADD', key, now, now)
  redis.call('PEXPIRE', key, window)
  return 1  -- ALLOWED
end
return 0  -- BLOCKED
```

VALKEY 9.0

Hash field expiration — 20% less memory.

Before 9.0, a token bucket for 1M users needed 2M keys — one per field. HEXPIRE changes that.

BEFORE v9.0

```
# 2 separate keys per user
SET rl:{user}:tokens   95
SET rl:{user}:refill   1717...

# 1M users = 2M keys
# Each key carries TTL metadata
# No atomic multi-field expiry
# Memory overhead: ~20% extra
```

→

VALKEY 9.0

```
# One hash per user
HSET rl:{user} tokens 95 refill 1717

# Per-field TTL — new in 9.0
HEXPIRE rl:{user} 3600 FIELDS 2
         tokens refill

# 1M users = 1M keys
# Fields expire independently
# 20% memory savings. Clean.
```

THE AI ANGLE

Request count is the wrong unit.

When one API call can cost 100× another,
your rate limiter must count tokens — not requests.

THE AI ANGLE

Token-aware limits for LLM APIs.

BROKEN MODEL

100 req / min

User A: 100 calls × 100 tokens
= 10,000 tokens, cheap

User B: 1 call × 100,000 tokens
= 100,000 tokens, 10× cost

Same limit. Wildly different bill.

TOKEN-AWARE

1M tokens / day

```
INCRBY rl:ta:{user}:used {n}
EXPIRE rl:ta:{user}:used 86400
```

```
Fair. Cost-bounded.
Flat key. Pipeline: atomic pair.
```

HYBRID (PRODUCTION)

req/min + tokens/day

Combine both strategies:

Burst protection req/min
Cost protection tokens/day
Model-tier limits per model

Two keys per user. One pipeline.

FAILURE MODES

Designing for when Valkey goes down.

This is a business decision, not a technical one. Make it explicit before you ship.

VALKEY UNREACHABLE	FAIL OPEN	FAIL CLOSED
→ *Public APIs → fail closed* *Internal services → fail open*	Allow request. Log. Alert on-call. Risk: users can exceed limits during Valkey downtime windows.	Return HTTP 503. Block everything. Risk: your service appears down even though the app server is healthy.
PARTIAL CLUSTER FAILURE	**FAIL OPEN**	**FAIL CLOSED**
→ *Upgrade to 9.0. This problem shrinks.*	WAIT with timeout, retry on another node. Use stale local cache for 5 seconds as fallback.	Return cached last-known limit (stale, but bounded). Valkey 9.0 atomic slot migration minimizes this entirely.

LIVE DEMO

Live demo: 25 lines that actually work.

What we're running

- **Sliding window**

  Atomic Lua. 5 req / 10s window. Watch it block then auto-reset.

- **Token bucket**

  Capacity=5, refill=1/sec. Burst 6 requests. Wait 3s. Try again.

- **Token-aware**

  Daily budget: 1,000 tokens. Four LLM calls. Third one pushes over.

```
docker run -p 6379:6379 valkey/valkey:latest
```

demo.py — run it live

```
limiter = ValkeyRateLimiter(
    host='localhost', port=6379
)

for i in range(8):
    result = limiter.sliding_window(
        user_id='user_123',
        limit=5, window_ms=10000
    )

    status = 'ALLOWED' if result.allowed \
             else 'BLOCKED'
    print(f"Req {i+1}: {status}")
    time.sleep(
0.5)
```

KEY TAKEAWAYS

01

Atomicity is non-negotiable.

Lua scripts give you check-and-increment in one round-trip. No locks, no races, no double-spend.

02

Valkey 9.0 HEXPIRE is a real upgrade.

Token buckets are now 20% leaner. Hash field TTL eliminates the separate-key-per-field antipattern.

03

AI era demands token-aware limits.

Request count is meaningless when one call can cost 1,000 tokens or 100,000. Count what costs money.

04

Design failure mode before launch.

Fail open vs fail closed is a product decision. Don't let the on-call engineer decide it at 2am.